

CyberSentinel - README.md

AI Cyber Threat Detection | PERN Stack | Team Code Wizards | SIH 2026

CyberSentinel - AI Cyber Threat Detection

This README is written so a beginner developer can rebuild a working clone from scratch by following it. It documents the actual code in backend/ + client/, not an ideal. Where code cuts corners for hackathon speed, it says so.

What: monitors network/user/device behaviour, correlates weak signals into high-confidence detections with explainable risk 0-100, opens incidents, remediates, reports.

Why: signature-only tools miss zero-days. 1 weak signal = noise, 3-4 correlated = attack.

How: PERN - PostgreSQL (Neon) + Express + React (Vite) + Node. Rule+heuristic correlation now, ML-pluggable later. No external AI API needed.

```
External App pushes signals --> POST /api/ingest --> Normalize --> Score --> Correlate
                                                                    |
                                                                    +-----+
                                                                    ?
detections + auto-incidents --> React UI
(Monitor / Graph / Detection / Incidents / Report)
```

1. Tech Stack (exact)

Backend backend/package.json:

node ESM ("type":"module"), express@4.19.2, pg@8.12, jsonwebtoken@9, bcryptjs@2.4.3, zod@3.23, helmet@7, cors@2.8, morgan@1.10, express-rate-limit@7, cookie-parser@1.4, dotenv@16, nanoid@5. Scripts: dev: node --watch server.js, start: node server.js, db:migrate: node src/db/migrate.js. Entry backend/server.js -> src/app.js, port from PORT (5000 in .env, code default 3001).

Frontend client/package.json:

react@19.3 + react-dom, react-router-dom@7.18, vite@6.3, @vitejs/plugin-react@4, tailwindcss@4 + @tailwindcss/vite, axios@1.20, recharts@3.10, vis-network@10.1, lucide-react@1.48, typescript@7. Scripts: dev: vite (:5173), build: tsc --noEmit && vite build. Alias @ -> client/ root (vite.config.ts). Dev proxy /api -> http://localhost:5000 (note: backend defaults to 3001 - set PORT=5000 in dev or fix proxy/VITE_API_URL).

DB: Postgres (Neon pooled URL ?sslmode=require, local Postgres works for dev). pg.Pool max 10, ssl rejectUnauthorized:false (src/config/db.js).

2. Actual Folder Structure (clone this)

```
backend/
  server.js          # entry: loads app+env, listen
  .env.example / .env # PORT, FRONTEND_URL, DATABASE_URL, JWT_*
  src/
    app.js           # helmet->cors->morgan->json->cookie->rateLimit->routes->404->errors
    config/env.js, config/db.js # env parse + pg Pool + withTransaction
```

```

middlewares/auth.middleware.js # authenticate: cookie OR Bearer -> req.user
middlewares/authorize.middleware.js # authorize(...roles) - EXISTS BUT UNUSED
middlewares/validate.middleware.js # zod -> req.validated
middlewares/error.middleware.js # notFound + errorHandler
validations/auth.validation.js # register/login schemas
validations/sentinel.validation.js # ingest/monitor/incident schemas
services/auth.service.js # hashPassword, signAccessToken, cookieOptions
services/normalize.service.js # BASE_RISK map + normalizeSignal()
services/aiScoring.service.js # scoreSignal(): base+boosts -> 0-100
services/correlation.service.js # PATTERNS + correlateBatch()
services/graph.service.js # buildGraph(orgId, hours)
controllers/auth.controller.js / ingest.controller.js / monitor.controller.js
controllers/detection.controller.js / incident.controller.js / report.controller.js
routes/auth.route.js / ingest.route.js / monitor.route.js / graph.route.js
routes/detection.route.js / incident.route.js / report.route.js
models/user.model.js, signal.model.js, detection.model.js, incident.model.js # thin, mostly unused
db/schema.sql, db/migrate.js
utils/apiResponse.js, utils/asyncHandler.js

client/
  vite.config.ts / index.html / .env.example # VITE_API_URL
  src/
    main.tsx # boots <App/>
    App.tsx # ContentProvider>AuthProvider>Router>SentinelLayout>Routes
    sentinel/
      api.ts # axios baseURL + TOKEN_KEY + fetch* helpers
      auth.ts # AuthProvider {user, loading, login, register, logout}
      layout.tsx # SentinelHeader nav + SentinelLayout wrapper
      ui.tsx # SeverityBadge, RiskBar, StatCard, Spinner
      pages/HomePage.tsx / MonitorPage.tsx / GraphPage.tsx / DetectionPage.tsx
      pages/IncidentsPage.tsx / IncidentDetailPage.tsx / ReportPage.tsx / AuthPages.tsx
      components/*, lib/*, styles/* # marketing template (Qronos) - NOT needed for clone

```

Old README described frontend/src/api/*, components/layout/* - that does not match code. Use the tree above.

3. Prerequisites + Quickstart (zero -> running)

```

# 0. Needs: Node 20+, Postgres (Neon free at console.neon.tech), git
# 1. Backend
cd backend && npm install
cp .env.example .env # then fill DATABASE_URL + JWT_SECRET (see §4)
npm run db:migrate # runs src/db/schema.sql
npm run dev # :5000 (matches vite proxy)

# 2. Frontend (new terminal, or npm run dev from repo root for both)
cd Client && npm install
cp .env.example .env # VITE_API_URL=http://localhost:5000 - empty = use /api proxy
npm run dev # :5173 -> open http://localhost:5173, Register a workspace

# 3. Verify
curl http://localhost:5000/api/health
# {"success":true,"data":{"status":"ok","service":"cybersentinel"}}

```

First run: open the app and Register - the first account creates your organization as OWNER -> lands on /monitor.

4. Environment Variables

backend/.env (src/config/env.js):

Key	Example	Notes
PORT	3001 (use 5000 to match client/vite.config.ts proxy)	Number(process.env.PORT 3001)
FRONTEND_URL	http://localhost:5173	CORS origin, credentials:true
DATABASE_URL	postgresql://owner:xxx@ep-xxx-pooler.../neondb?sslmode=require	required, pooled Neon URL
JWT_SECRET	node -e "console.log(require('crypto').randomBytes(64).toString('hex'))"	signs 1h access JWT
JWT_REFRESH_SECRET	same gen	stored in sessions.refresh_token (30d)
JWT_EXPIRES_IN	1h	passed to jwt.sign
TOKEN_COOKIE_NAME	cybersentinel-token	HttpOnly cookie name

client/.env:

Key	Example	Notes
VITE_API_URL	http://localhost:3001 (backend URL, no trailing slash). Empty = same-origin /api via Vite proxy.	

Code: `baseURL = VITE_API_URL ? VITE_API_URL + '/api' : '/api'` (sentinel/api.ts:7-10).

CORS + cookies require `withCredentials:true` (already set) and matching `FRONTEND_URL`.

5. Database (clone exactly)

Run `backend/src/db/schema.sql` via `npm run db:migrate`. Core tables (all org-scoped by `organization_id` ON DELETE CASCADE):

- `organizations(id TEXT pk gen_random_uuid(), name, slug UNIQUE, created_at);` `users(id, email UNIQUE, password_hash, first_name, last_name, organization_id FK, last_login_at);` `organization_members(id, role DEFAULT 'MEMBER', user_id FK, organization_id FK, UNIQUE(user,org));` `sessions(id, user_id FK, refresh_token UNIQUE, ip, user_agent, expires_at).`
- `devices(id, hostname, os, ip_address, criticality DEFAULT 50, organization_id FK, last_seen_at).`
- `signals(id, signal_type, category, severity DEFAULT 'INFO', message, source_ip, user_identity, hostname, domain, raw_data JSONB DEFAULT '{}', risk_score INT, risk_factors TEXT[], organization_id FK, device_id FK NULL, event_timestamp TIMESTAMPTZ, ingested_at) + idx_signals_org_time(org,time), idx_signals_type.`
- `network_events(...)` - created but unused by controllers.
- `detections(id, title, detection_type, risk_score, confidence, severity, explanation JSONB, status DEFAULT 'OPEN', organization_id FK);` `detection_signals(detection_id FK CASCADE, signal_id FK CASCADE, weight FLOAT, PK pair).`
- `incidents(id, title, description, status DEFAULT 'OPEN', severity, assignee_id FK users NULL, detection_id FK detections NULL, organization_id FK, created_at, resolved_at) + idx_incidents_org_status;` `remediation_logs(id, incident_id FK CASCADE, user_id FK users, action, notes, created_at).`

6. Backend API (request/response to clone)

Base: `helmet + cors(origin:FRONTEND_URL) + morgan(dev) + json(1mb) + cookieParser + rateLimit 200/15min global, 30/15min /api/auth` (src/app.js:26-50). Mounts: `/api/auth, /api/ingest, /api/monitor, /api/graph, /api/detections, /api/incidents, /api/reports`. Shape: `{success:true,data,...}` or `{success:false,error}` (utils/apiResponse.js). All except `register/login/health` need `authenticate` (cookie `cybersentinel-token` or `Authorization: Bearer -> req.user{id,email,firstName,lastName,organizationId,organization,role}` via DB join, `COALESCE(role,'OWNER')`).

Method	Endpoint	Body/Query	Returns
--------	----------	------------	---------

```

|---|---|---|---|
| POST | /api/auth/register | {email,password?8,firstName,lastName,organizationName} |
{user,organization,role:'OWNER',token} + HttpOnly cookie. Creates org slug, bcrypt-12, member OWNER, session row. 409
if email exists. |
| POST | /api/auth/login | {email,password} | {user,organization,role,token,refreshToken} + cookie, updates
last_login_at. 401 if bad. |
| GET | /api/auth/session | JWT | req.user |
| POST | /api/auth/logout | JWT | clears cookie (does not delete session row) |
| POST | /api/ingest/ |
{signals:[{signalType,category:NETWORK\|AUTH\|ENDPOINT\|EMAIL\|WEB,severity:INFO\|CRITICAL,message,sourceIp?,userIdent
ity?,hostname?,domain?,deviceId?,rawData:{},eventTimestamp:ISO}}] 1-500 |
{processed,detectionsTriggered,incidentsCreated}. Requires JWT (not API key - see §7). |
| GET | /api/monitor/signals?page&limit&severity&category&search&minRisk | query |
{success,data:Signal[],meta:{page,limit,total,totalPages}} ordered event_timestamp DESC, ILIKE search on
message/type/user. |
| GET | /api/monitor/stats | - | {totalSignals,detections24h,openIncidents(OPEN\|INVESTIGATING),devices} |
| GET | /api/detections?type&severity | query | paginated detections |
| GET | /api/detections/:id | - | detection + signals[]+weight |
| POST | /api/detections/:id/promote | - | creates OPEN incident from detection |
| GET | /api/incidents?status&severity | query | paginated incidents |
| GET | /api/incidents/:id | - | incident + remediation_logs + linked signals via detection |
| PATCH | /api/incidents/:id | {status:OPEN\|INVESTIGATING\|CONTAINED\|RESOLVED\|FALSE_POSITIVE\|DISMISSED,
assigneeId?} | updated incident, sets resolved_at=NOW() on RESOLVED |
| POST | /api/incidents/:id/remediation | {action,notes?} | new remediation_log |
| GET | /api/graph?window=1h\|24h\|7d | query |
{nodes:{id,type:user\|device\|ip\|domain,label,risk}[],edges:{from,to,label}[]} built from last N hours signals LIMIT
500, edges capped 800 (services/graph.service.js). |
| GET | /api/reports?start&end | ISO query |
{range,byType[{detection_type,c}],bySeverity[{severity,c}],topRiskyUsers[{user_identity,avg_risk,c}],incidents[{status
,c}]} |

```

Curl ingest (replace TOKEN):

```

curl -X POST http://localhost:3001/api/ingest \
-H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
-d '{"signals":[
    {"signalType":"PHISH_CLICK","category":"EMAIL","severity":"MEDIUM","message":"Clicked
xn--paypa1.com","userIdentity":"adi@corp.com","eventTimestamp":"2026-09-26T10:00:00Z","rawData":{}},
    {"signalType":"CREDENTIAL_FORM_POST","category":"WEB","severity":"HIGH","message":"Cred
submit","userIdentity":"adi@corp.com","eventTimestamp":"2026-09-26T10:02:00Z","rawData":{}},
    {"signalType":"IMPOSSIBLE_TRAVEL","category":"AUTH","severity":"HIGH","message":"IN then NG
12min","userIdentity":"adi@corp.com","sourceIp":"103.21.244.10","eventTimestamp":"2026-09-26T10:14:00Z","rawDat
a":{}}
]}'
# -> PHISHING ~85 -> auto Incident (score>=70)

```

7. Ingest -> Detection Pipeline (the main workflow to clone)

POST /api/ingest (controllers/ingest.controller.js:9-57):

- 1. Normalize (services/normalize.service.js): signalType.toUpperCase(), BASE_RISK e.g. SHADOW_COPY_DELETE:55, PRIV_ESCALATION:50, IMPOSSIBLE_TRAVEL:50, BRUTE_FORCE:45, CREDENTIAL_FORM:45, MASS_RENAME:45, HIGH_ENTROPY:40, BEACONING:40, default:15.
- 2. Score (services/aiScoring.service.js): score=base +10 if deviceCriticality>=80 +10 if CRITICAL +5 if HIGH +15 threat-intel IP/domain, clamp 0-100, risk_factors[] strings. Note: ingest calls scoreSignal(n,{}) so intel/criticality boosts

never fire today - wire device criticality + badIps to enable.

- 3. Insert signals with organization_id=req.user.organizationId.
- 4. Correlate (services/correlation.service.js): group batch by org[user]host; patterns:
 - RANSOMWARE: MASS_FILE_RENAME .35 + SHADOW_COPY_DELETE .35 + HIGH_ENTROPY .2 + OUTBOUND_TOR .1
 - PHISHING: PHISH_CLICK .3 + CREDENTIAL_FORM .3 + IMPOSSIBLE_TRAVEL .25 + MFA_FAILURE .15
 - MALWARE: RARE_PROCESS .3 + BEACONING .3 + SUSPICIOUS_DNS .2 + NEW_ASN .2
 - UNAUTH_ACCESS: BRUTE_FORCE .3 + OFF_HOURS .2 + PRIV_ESCALATION .3 + SENSITIVE_SHARE .2
 Require ?2 distinct OR weight?.35, score=round(sum*100), drop <55. severity: ?85 CRITICAL, ?70 HIGH, ?40 MEDIUM. confidence=min(95,55+12*n). explanation={matchedSignals,weights,reasoning}.
- 5. Persist detections + detection_signals; if score>=70 auto incidents OPEN.

Limitation to know for clone: correlates only current request batch, not 30-min DB window (comment says 30-min but code doesn't query history). To fix, query signals WHERE org+user/host AND event_timestamp > NOW()-30min before scoring.

How external app integrates: customer registers org -> embeds fetch POST /api/ingest in their backend where logins/file/email events happen -> we score/correlate -> they view results in our React UI. We are push-only; no pull/agent.

8. Frontend (clone exactly)

client/src/App.tsx:29-52: ContentProvider > AuthProvider > Router > SentinelLayout > Routes: /->HomePage, /login->LoginPage, /register->RegisterPage, /monitor->MonitorPage, /graph->GraphPage, /detection->DetectionPage, /incidents->IncidentsPage, /incidents/:id->IncidentDetailPage, /report->ReportPage, *=->/. Protected via RequireAuth (spinner while loading, else Navigate /login).

sentinel/api.ts: axios baseUrl=VITE_API_URL+'/' else '/api', withCredentials:true, request interceptor Bearer localStorage cybersentinel-token. Helpers
fetchStats/Signals/Graph/Detections/Detection/promote/Incidents/Incident/updateIncident/addRemediation/Report
unwrap r.data.data (paginated returns r.data with meta).

sentinel/auth.tsx:

AuthProvider{user:SessionUser{id,email,firstName,lastName,organizationId,organization{id,name,slug},role},loading,login,register,logout}. Mount: if token GET /auth/session else loading=false. login/register -> setToken+setUser -> navigate /monitor. logout -> POST /auth/logout + clear.

Pages ? API:

- HomePage: hero + 4 threat cards + GET /monitor/stats -> StatCards.
- MonitorPage: 5s polling (page 1 only), filters search/severity/category/minRisk, GET /monitor/signals.
- GraphPage: vis-network, window 1h/24h/7d, GET /graph?window=, node size=risk color=type, click->detail.
- DetectionPage: GET /detections?type, GET /detections/:id (reasoning+weights), POST /detections/:id/promote.
- IncidentsPage/Detail: GET /incidents?status, GET /incidents/:id, PATCH {status}, POST .../remediation {action,notes}.
- ReportPage: GET /reports?start&end, recharts Bar/Pie + risky-users table + JSON export.
- AuthPages: Login prefill demo, Register org+names+password?8.

sentinel/layout.tsx: fixed header CYBERSENTINEL Home|Monitor|Graph|Detection|Incidents|Report + email/logout, mobile hamburger. sentinel/ui.tsx: badges, RiskBar, StatCard.

9. Roles & Workflow

- Employee (no login): generates signals, appears as user_identity / topRiskyUsers.
- Analyst/Member (login): Home->Monitor->Graph->Detection->Promote->Incidents(assign/CONTAINED/RESOLVED + BLOCKED_IP/ISOLATED_HOST/...)->Report.
- Owner/Admin: stored as organization_members.role (OWNER on register) but not enforced - authorize() exists (middlewares/authorize.middleware.js) but no route uses it. Any authed user can ingest/promote/remediate. Add authorize('OWNER') to routes to enforce. Tenant isolation is by organization_id in every query.

Demo (2 min): Home(score,critical) -> Monitor(phish feed) -> Graph(user->evil domain) -> Detection(why 88) -> Promote -> Incidents(assign+isolate) -> Report.

10. Deploy + Troubleshoot

- Frontend Vercel: set VITE_API_URL=https://<backend>, npm run build -> dist/. Backend Railway/Render: set PORT, FRONTEND_URL=https://<vercel>, DATABASE_URL, JWT_*, npm run db:migrate, npm start. DB Neon.
- Pitfalls: backend PORT must match the frontend target (5000 in both .env files by default); CORS fails if FRONTEND_URL wrong; 401 -> missing Bearer/cookie or expired 1h JWT -> re-login; 429 -> rate limit; empty graph/report -> no signals in window -> ingest via POST /api/ingest first.

11. Glossary

Signal=single event; Detection=correlated signals+risk; Incident=human case; Risk 0-39 LOW 40-69 MED 70-84 HIGH 85-100 CRIT (auto-incident ?70); MTTR=mean resolve time; Graph=nodes user/device/ip/domain + edges.